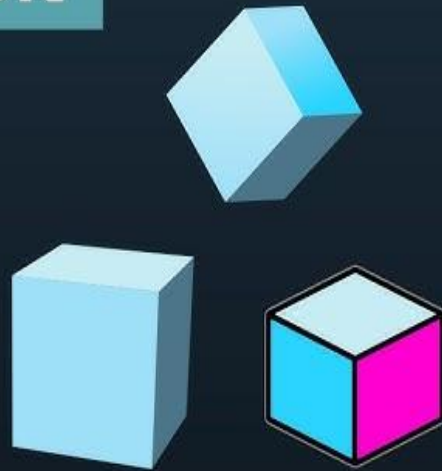


PROGRAMACIÓN

ORIENTADA

A OBJETOS



Programación Orientada a Objetos JavaScript

Desarrollo Entorno de Cliente

PabloSG

Índice

1.Título del tema	2
2.Explicación detallada del contenido	2
2.1 Qué es la POO en JavaScript	2
2.1.1 Class	2
2.1.2 Constructor	3
2.1.3 Subclases	3
2.1.4 Variables	3
2.1.5 New.....	3
2.2 Problemas comunes	4
2.3 Similitud con la vida real.....	4
2.4 Ejemplos prácticos	4
Ejemplo 1: Crear una clase	4
Ejemplo 2: Crear un objeto.....	4
3.Resumen	5

1.Título del tema

Introducción de la Programación Orientada a Objetos en JavaScript

2.Explicación detallada del contenido

La Programación Orientada a Objetos (POO) organiza el código alrededor de objetos que combinan datos (propiedades) y acciones (métodos). En JavaScript, la POO permite escribir programas más claros, reutilizables y fáciles de mantener, especialmente a medida que el proyecto crece.

- **Orientación a objetos:** Representa entidades de la vida real con propiedades y métodos.
- **Palabras clave:** class, constructor, new, extends, this, super.
- **Problemas comunes:** Código desordenado con funciones y variables sueltas.
- **Similitud con la vida real:** Modelar personas, coches, animales... cada uno con características y acciones.
- **Ejemplos:** Clases, herencia, encapsulación, patrones de diseño y uso práctico.

2.1 Qué es la POO en JavaScript

La Programación Orientada a Objetos (POO) es un estilo de programación que utiliza estructuras llamadas objetos para representar elementos de la vida real.

JavaScript permite crear y usar objetos para organizar el código, dividir tareas y mantener todo ordenado.

La POO mejora la claridad, la estructura y la reutilización del código.

2.1.1 Class

La palabra clave `class` se usa para crear un molde

Ese molde sirve para construir objetos.

Ejemplo:

```
class Persona {  
  // Aquí se irá añadiendo propiedades y métodos  
}
```

Es como un plano para crear personas: describe cómo serán, pero no es la persona en sí.

2.1.2 Constructor

Dentro de una clase, el constructor es un bloque que se ejecuta cuando se crea un objeto que sirve para darle valores al nuevo objeto.

Ejemplo:

```
class Persona {  
  // El constructor recibe parámetros y asigna valores al objeto  
  constructor(nombre, edad) {  
    this.nombre = nombre; // 'this' se refiere al objeto actual  
    this.edad = edad;      // Guardamos la edad en la propiedad del objeto  
  }  
}
```

2.1.3 Subclases

Una subclase es una clase que hereda de otra, que se hace con extends.

Ejemplo:

```
class Estudiante extends Persona {  
  estudiar() {  
    console.log(this.nombre + " está estudiando");  
  }  
}
```

Es como decir: “Estudiante es un tipo de Persona”.

2.1.4 Variables

Las variables guardan información.

Ejemplo:

```
// Declaramos variables simples  
const persona1 = new Persona("Carlos", 28);
```

En POO, estas variables suelen estar dentro de un objeto o clase, en lugar de estar sueltas.

2.1.5 New

new sirve para crear un objeto nuevo.

Ejemplo:

```
// Creamos un objeto de la clase Persona usando 'new'  
const persona = new Persona("Ana", 20);
```

Es como decir: “crea una nueva persona”.

2.2 Problemas comunes

Cuando no organizamos bien el código:

- Hay demasiadas variables sueltas
- Hay funciones por todos lados
- Es difícil encontrar dónde cambiar algo
- Todo se vuelve confuso

La POO ayuda a ordenar todo dentro de objetos y clases.

2.3 Similitud con la vida real

La POO funciona como en la vida real:

- Una Persona tiene nombre, edad y puede hablar
- Un Coche tiene marca, modelo y puede arrancar
- Un Animal tiene tipo y puede hacer sonidos

Cada cosa tiene características y acciones; así funcionan los objetos.

2.4 Ejemplos prácticos

Ejemplo 1: Crear una clase

```
class Persona {  
  // El constructor inicializa las propiedades del objeto  
  constructor(nombre, edad) {  
    this.nombre = nombre; // Propiedad nombre  
    this.edad = edad;     // Propiedad edad  
  }  
  
  // Método que hace que la persona salude  
  saludar() {  
    console.log("Hola, soy " + this.nombre);  
  }  
}  
  
const persona1 = new Persona("Carlos", 28);  
persona1.saludar(); // Hola, soy Carlos
```

Ejemplo 2: Crear un objeto

```
const estudiante1 = new Estudiante("Lucía", 18);  
estudiante1.saludar(); // Hola, soy Lucía  
estudiante1.estudiar(); // Lucía está estudiando
```

3.Resumen

- La POO usa objetos para organizar el código.
- Una clase es un molde.
- Un objeto es algo creado con ese molde.
- El constructor inicializa los valores.
- new crea nuevos objetos.
- Las subclases permiten heredar características.
- Es útil para tener el código ordenado y fácil de entender.